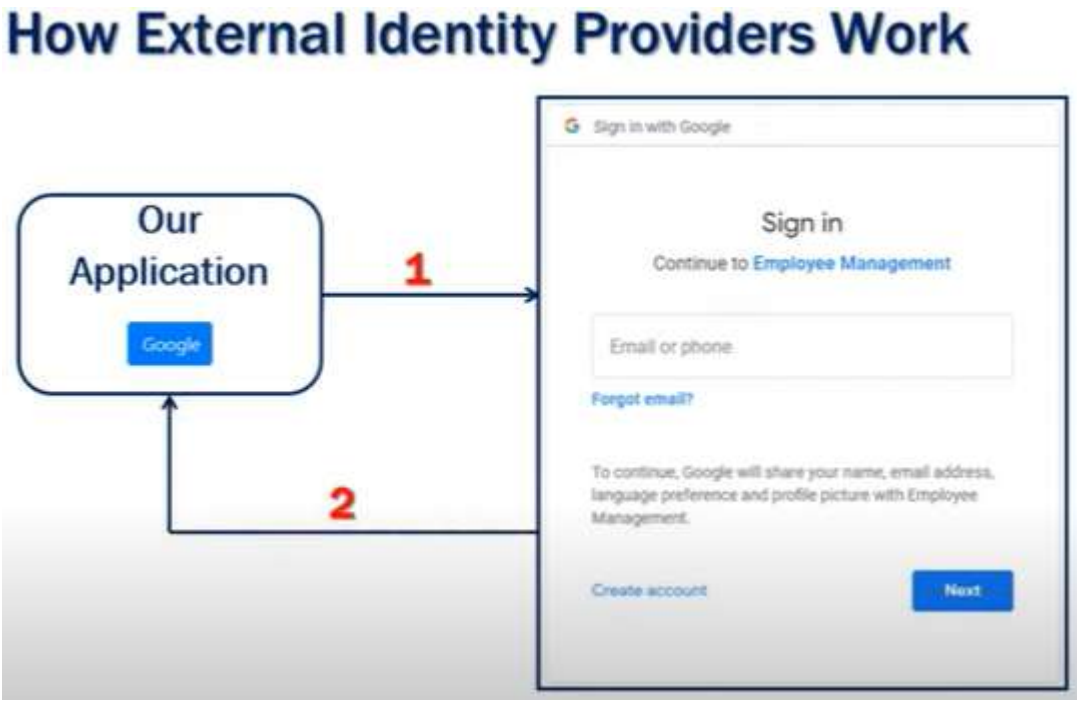
# 107 - ExternalLoginCallback action




AccountController.cs | LoginViewModel.cs

EmployeeManagement.Controllers.AccountControlle | ExternalLoginCallback(string returnUrl = null, string

```
[AllowAnonymous]
public async Task<IActionResult>
    0 references
    ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{       if returnUrl is null, we initialize it to root URL
    returnUrl = returnUrl ?? Url.Content("~/");

    LoginViewModel loginViewModel = new LoginViewModel
    {    otherwise it will have the value of the incoming returnUrl
        ReturnUrl = returnUrl,
        ExternalLogins =
                (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };
    we check the second incoming parameter:
    if (remoteError != null)  we received some error
    {
        ModelState
            .AddModelError(string.Empty, $"Error from external provider: {remoteError}");
              redirect back to        + user can see the error received from the external provider
        return View("Login", loginViewModel);
    }
```

```
// Get the login information about the user from the external login provider
var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null) we don't have info
{
    ModelState add this error message to the ModelState
        .AddModelError(string.Empty, "Error loading external login information.");

    return View("Login", loginViewModel);  redirect to Login + error message to user
}
```

```
signInManager.GetExternalLoginInfoAsync();
                                                    Search Solution Explorer (Ctrl+;)
(awaitable) Task<ExternalLoginInfo> SignInManager<ApplicationUser>.GetExternalLoginInfoAsync([string expectedXsrf = null])
Gets the external login information for the current login, as an asynchronous operation.
Error(string.Empt
```

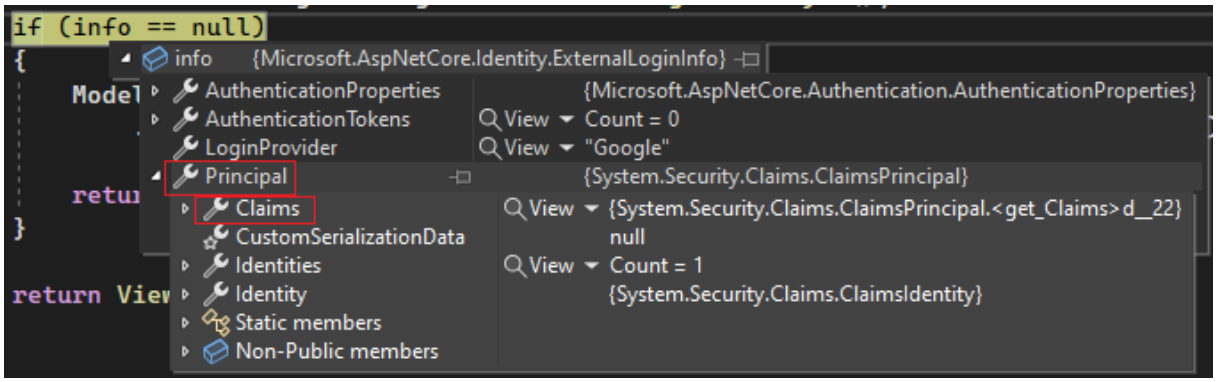
Обръщаме внимание, че от Intellisense, метод .GetExternalLoginInfoAsyc() връща обект от тип <ExternalLoginInfo>. За да разберем какво означава резултат от такъв тип, нека поставим BreakPoint.

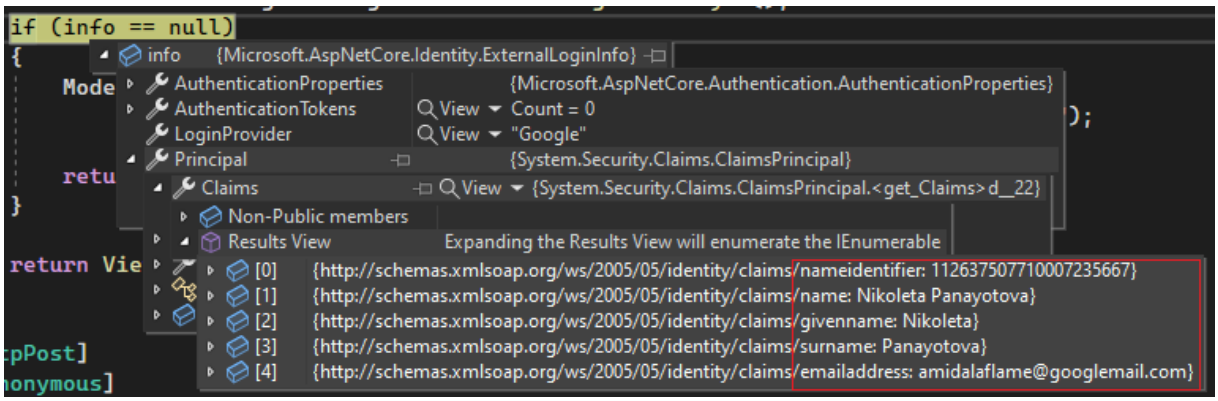
```
110     [AllowAnonymous]
111     public async Task<IActionResult>
            0 references
112     ExternalLoginCallback(string returnUrl = null, string remoteError = null)
113     {                          the return type of this method is not satisfied
114         returnUrl = returnUrl ?? Url.Content("~/");
115
116         LoginViewModel loginViewModel = new LoginViewModel...;
122
123         if (remoteError != null)...
130
131         // Get the login information about the user from the external login provider
132         var info = await signInManager.GetExternalLoginInfoAsync();
133         if (info == null)
134         {
135             ModelState
136                 .AddModelError(string.Empty, "Error loading external login information.");
137
138             return View("Login", loginViewModel);
139         }
140       to fix this 1 x error add this:
141       return View("Login", loginViewModel); in general section temporarily
142     }
110 %   1   0                                                    Ln: 141   Ch: 13
```

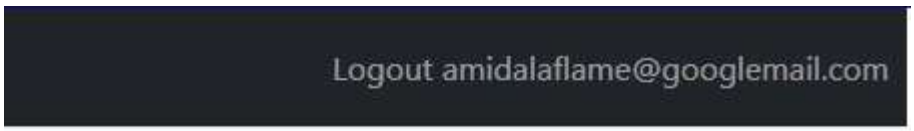
При успешно вписване, имаме пълен набор информация за потребителя:

```
133         if (info == null)
134         {                  info  {Microsoft.AspNetCore.Identity.ExternalLoginInfo}
135             Mod              AuthenticationProperties   {Microsoft.AspNetCore.Authentication.AuthenticationProperties}
136                              AuthenticationTokens   View  Count = 0
137                              LoginProvider          View  "Google"
138             ret              Principal                    {System.Security.Claims.ClaimsPrincipal}
139         }                    ProviderDisplayName    View  "Google"
                                 ProviderKey            View  "112637507710007235667"
110 %   No issues found                                                    Ln: 133
```

ProviderKey е уникална стойност за дадения потребител от External login provider.

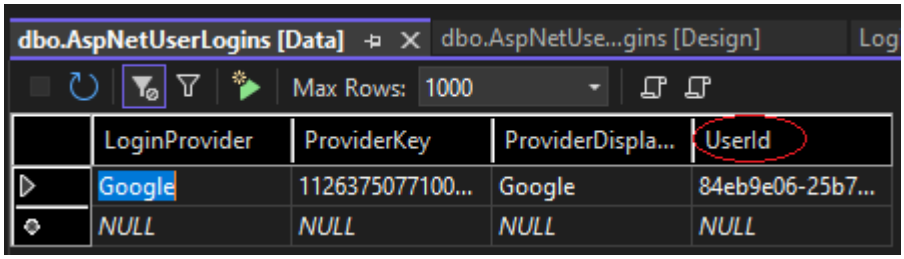




Ако нямаме успешно вписване, таблицата ще е празна, но аз направих едно предварително.




UsesrID from AslNetUserLogins таблицата ще кореспондира автоматично с Id на AspNetUsers и ще я попълни, така че независимо колко локални или външни вписвания има един потребител, той да е уникален.

dbo.AspNetUsers [Data] | dbo.AspNetUsers [Design] | dbo.AspNetUserLogins [Data]

Max Rows: 1000

| Id | UserName | NormalizedUs... | Email | NormalizedEm... | EmailConfirmed | Passwor |
|---|---|---|---|---|---|---|
| 1f9ea9da-40d8-... | aurora@abv.bg | AURORA@ABV.... | aurora@abv.bg | AURORA@ABV.... | False | AQAAAA |
| 84eb9e06-25b7... | amidalaflame@... | AMIDALAFLAM... | amidalaflame@... | AMIDALAFLAM... | False | NULL |
| 87cabcf7-6319-... | sara@abv.bg | SARA@ABV.BG | sara@abv.bg | SARA@ABV.BG | False | AQAAAA |
| a5b9400f-45cf-... | george@abv.bg | GEORGE@ABV.... | george@abv.bg | GEORGE@ABV.... | False | AQAAAA |
| b14a1adf-9835-... | sofia@abv.bg | SOFIA@ABV.BG | sofia@abv.bg | SOFIA@ABV.BG | False | AQAAAA |
| b2056455-c2b4... | bob@abv.bg | BOB@ABV.BG | bob@abv.bg | BOB@ABV.BG | False | AQAAAA |
| cf4120a9-5bca-... | david@abv.bg | DAVID@ABV.BG | david@abv.bg | DAVID@ABV.BG | False | AQAAAA |
| d2f456bd-46fc-... | tim@abv.bg | TIM@ABV.BG | tim@abv.bg | TIM@ABV.BG | False | AQAAAA |
| f86605f9-0791-... | jason@abv.bg | JASON@ABV.BG | jason@abv.bg | JASON@ABV.BG | False | AQAAAA |
| NULL | NULL | NULL | NULL | NULL | NULL | NULL |

Ако този External Login info object is not null, that means we have received login information from the external login provider (Google) we want to sign the user in using this provider so for that we are using this .ExternalSignInAsync method of the signInManager service. And to this method we pass the LoginProvider. Remember: on the external login for objective log and provider property returns our current external login provider and we have seen that value is Google at the moment. And then the ProviderKey, which is the unique identifier for the user, provided by the external login provider (in this case Google). And then we want to create a persistent cookie and if we want to bypassTwoFactor authentification. For this ExternalLoginSignAsync method to succeed, we have to have a corresponding record in aspNetUserLogins table.

```
// If the user already has a login (i.e if there is a record in AspNetUserLogins
// table) then sign-in the user with this external login provider
var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvider,
    info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
```

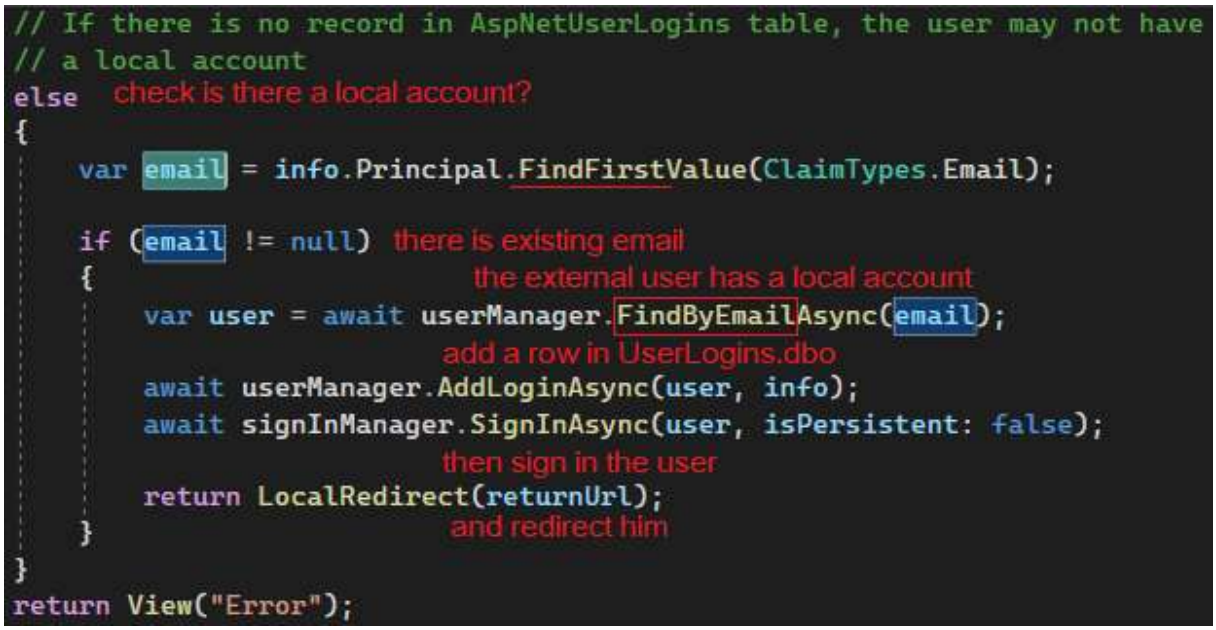
pass to
userId
cookie
authentification
dbo.AspNetUserLogins existing record

For a given LoginProvider, with a given ProviderKey, if we do not have a corresponding row in this AspNetLogins table, then most likely this external user might not have a corresponding local user account and local user accounts are in AspNetUserLogins table. So the row in aspNetUserLogins table is linked to a row in aspNetUsers table using this UserId column.

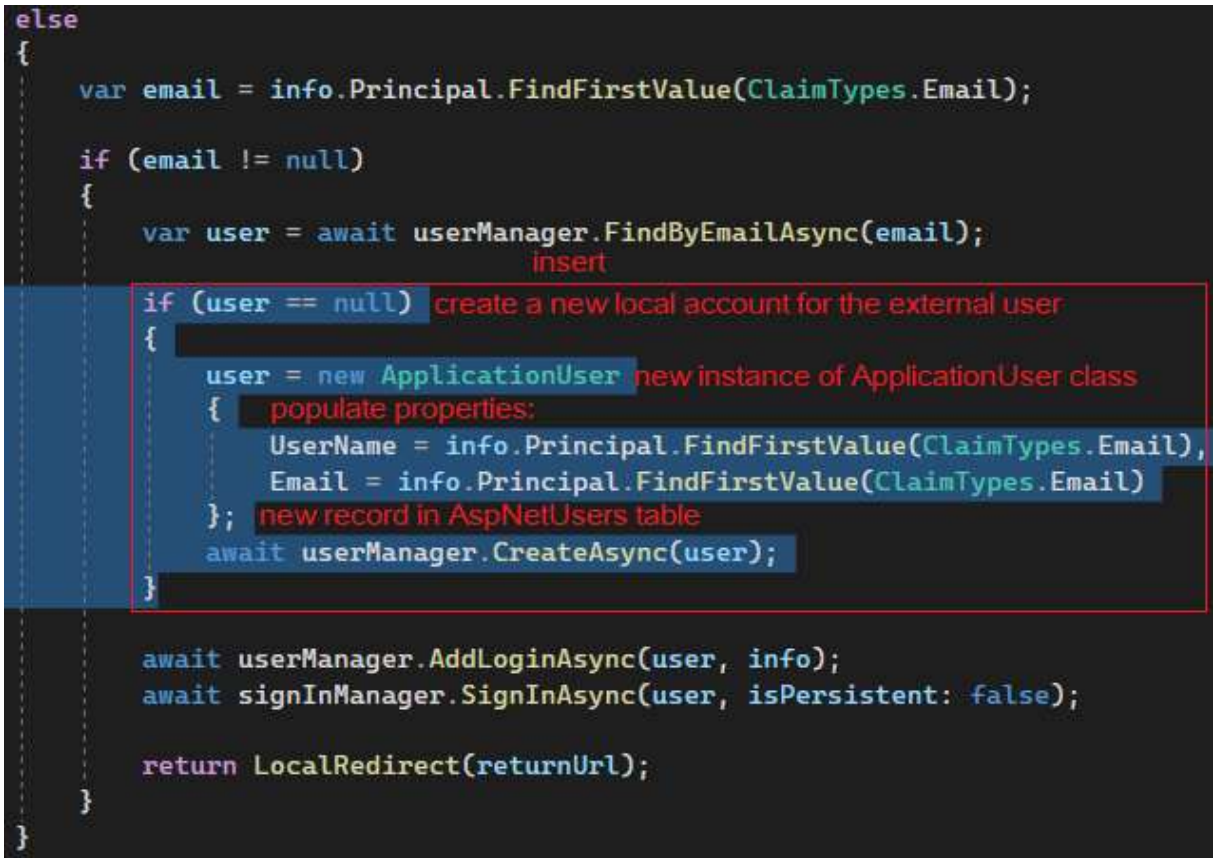
Ако нямаме запис на външен акаунт в AspNetLogins, по всяка вероятност липсва и кореспондиращ акаунт на локален потребител в AspNetUses. Тогава няма да знаем кой е външният потребител и метод ExternalLoginSingInAsync няма да е Succeed и ще търсим друго решение.

```
UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
Email = info.P                                    Types.Email)
```

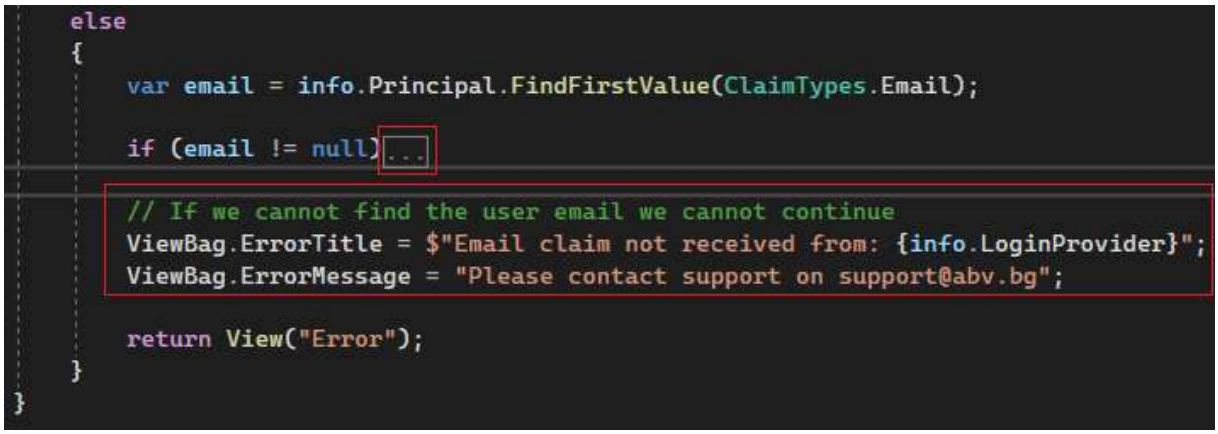
(local variable) ExternalLoginInfo info




```
            // If there is no record in AspNetUserLogins table, the user may not
have
            // a local account
            else
            {
                var email = info.Principal.FindFirstValue(ClaimTypes.Email);

                if (email != null)
                {
                    var user = await userManager.FindByEmailAsync(email);

                    await userManager.AddLoginAsync(user, info);
                    await signInManager.SignInAsync(user, isPersistent: false);

                    return LocalRedirect(returnUrl);
                }
            }
            return View("Error");
```




```
                    if (user == null)
                    {
                        user = new ApplicationUser
                        {
                            UserName =
info.Principal.FindFirstValue(ClaimTypes.Email),
                            Email =
info.Principal.FindFirstValue(ClaimTypes.Email)
                        };
                        await userManager.CreateAsync(user);
                    }
```



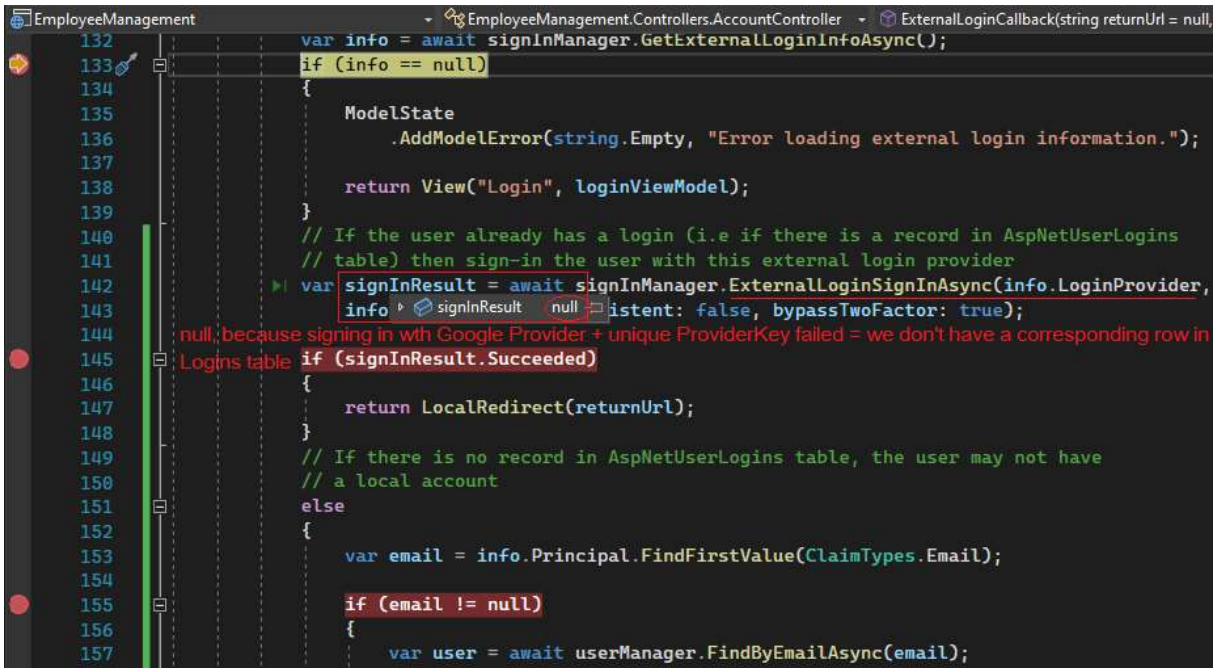

```
                // If we cannot find the user email we cannot continue
                ViewBag.ErrorTitle = $"Email claim not received from:
{info.LoginProvider}";
                ViewBag.ErrorMessage = "Please contact support on
support@abv.bg";
```

Full code:

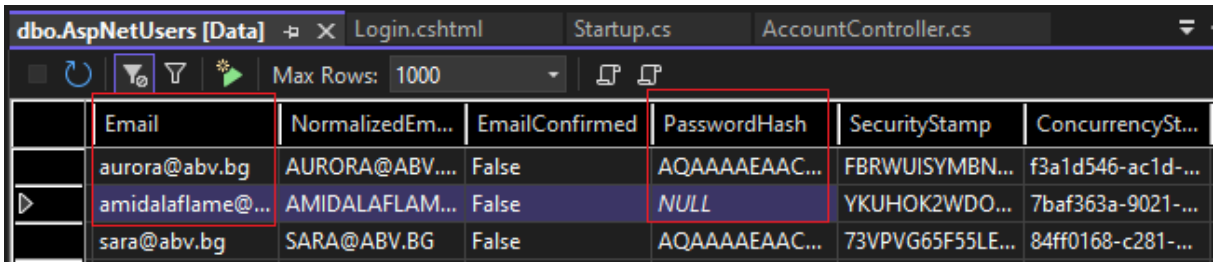
```
        [AllowAnonymous]
        public async Task<IActionResult>
            ExternalLoginCallback(string returnUrl = null, string remoteError =
null)
        {
            returnUrl = returnUrl ?? Url.Content("~/");

            LoginViewModel loginViewModel = new LoginViewModel
            {
                ReturnUrl = returnUrl,
                ExternalLogins =
                        (await
signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
            };

            if (remoteError != null)
            {
                ModelState
                    .AddModelError(string.Empty, $"Error from external provider:
{remoteError}");

                return View("Login", loginViewModel);
            }

            // Get the login information about the user from the external login
provider
            var info = await signInManager.GetExternalLoginInfoAsync();
            if (info == null)
            {
                ModelState
                    .AddModelError(string.Empty, "Error loading external login
information.");

                return View("Login", loginViewModel);
            }
            // If the user already has a login (i.e if there is a record in
AspNetUserLogins
            // table) then sign-in the user with this external login provider
            var signInResult = await
signInManager.ExternalLoginSignInAsync(info.LoginProvider,
                info.ProviderKey, isPersistent: false, bypassTwoFactor: true);

            if (signInResult.Succeeded)
            {
                return LocalRedirect(returnUrl);
            }
            // If there is no record in AspNetUserLogins table, the user may not
have
            // a local account
            else
            {
                var email = info.Principal.FindFirstValue(ClaimTypes.Email);

                if (email != null)
                {
                    var user = await userManager.FindByEmailAsync(email);

                    if (user == null)
                    {
                        user = new ApplicationUser
                        {
```

```
                            UserName =
info.Principal.FindFirstValue(ClaimTypes.Email),
                            Email =
info.Principal.FindFirstValue(ClaimTypes.Email)
                        };
                        await userManager.CreateAsync(user);
                    }

                    await userManager.AddLoginAsync(user, info);
                    await signInManager.SignInAsync(user, isPersistent: false);

                    return LocalRedirect(returnUrl);
                }

                // If we cannot find the user email we cannot continue
                ViewBag.ErrorTitle = $"Email claim not received from:
{info.LoginProvider}";
                ViewBag.ErrorMessage = "Please contact support on
support@abv.bg";

                return View("Error");
            }
        }
```

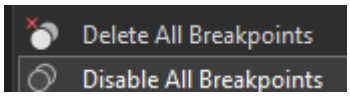
Debug -> Google ->



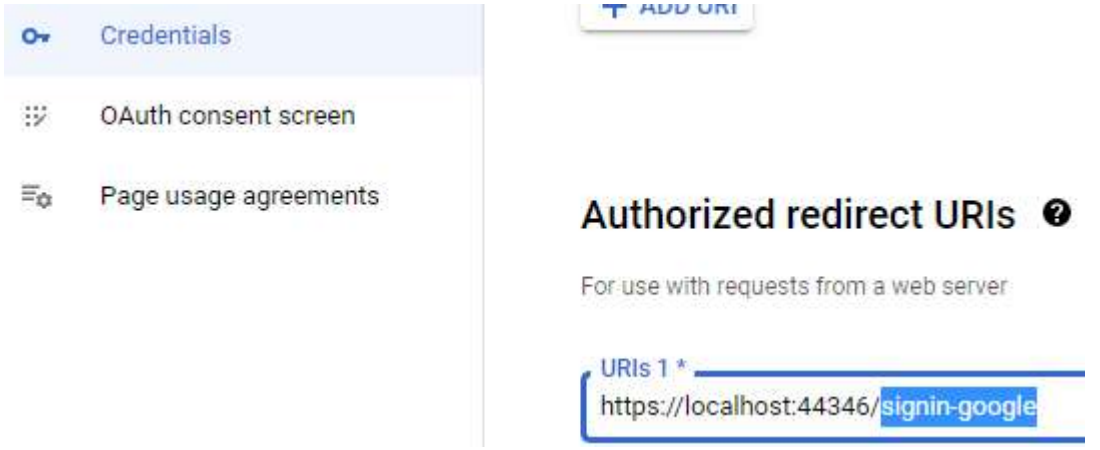

Обръщаме внимание, че паролата не се запазва при нас – тя е от външен потребител:



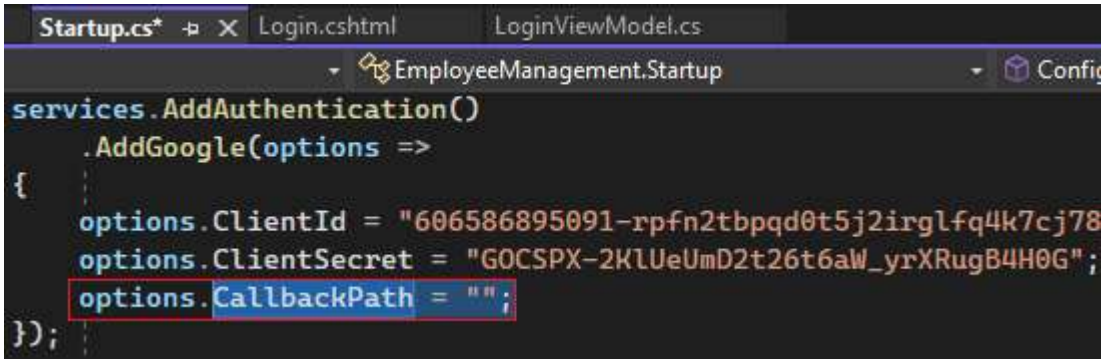

Имаме възможност за Debug -> Disable All BreakPoints:




По подразбиране, CallBack URL: https://localhost:44346/signin-google




Ако искаме да го сменим, това става от Startup:




```
options.CallbackPath = "";
```